

=====

MQ101 — Métodos Quantitativos para Políticas Públicas

Lista de Exercícios #01 — TEMPLATE DE SCRIPT (R)

Nome: Ana Carolina Wolff

RA: 23202510326

Turma: PPU-009 - MÉTODOS QUANTITATIVOS - TPPU00920253
(2025.3 - 4T234)

Data: 24/09/2025

Descrição: Fundamentos de R para Análise de Dados

=====

===== ORIENTAÇÕES =====

- 1) Este script é o MODELO para a sua entrega (.R).
- 2) Execute o script de cima para baixo (Ctrl/Cmd + Shift + Enter no RStudio)

ou use o botão “Source”.
- 3) Onde estiver escrito TODO ou ENTREGA, substitua pelos seus códigos e respostas.
- 4) Use comentários explicativos: diga o que o código faz e por que isso é útil.
- 5) Use `set.seed()` quando fizer simulações para garantir reprodutibilidade.
- 6) Gere também um PDF com os resultados e interpretações (via Rmd ou outro).
- 7) Não utilize pacotes além de ‘ggplot2’ (opcional). Prefira base R.

Se estiver no RStudio, esta chamada retorna informações do RStudio (pode falhar fora do RStudio).

```
tryCatch(RStudio.Version()$mode, error = function(e) “RStudio não detectado”)
```

Reprodutibilidade global para esta lista (você pode mudar, mas mantenha constante)

```
set.seed(202501)
```

para instalar pacote

```
install.packages (“ggplot2”) # Carregamento opcional do ggplot2 (apenas se desejar usar ggplot para gráficos)
if (requireNamespace(“ggplot2”, quietly = TRUE)) { library(ggplot2) } else { message(“Pacote ‘ggplot2’ não encontrado. Usando gráficos base R (hist).”) }
```

Dica: escolha uma pasta de trabalho, se necessário (descomente e ajuste):

```
setwd("~/caminho/para/sua/pasta")
```

=====

===== **EXERCÍCIO 1 (10 pts)** =====

R como calculadora — ordem das operações (HOPR cap. 1)

Objetivo: executar expressões e entender a ordem das operações.

ENTREGA: cole os resultados no PDF e escreva, em 3–5 linhas, como os parênteses afetam o resultado.

TODO: Execute as linhas a seguir e observe os resultados.

```
10 + 2 (10 + 2) * 3 ((10 + 2) * 3 - 6) / 3
```

```
#> 10 + 2 # [1] 12 #> (10 + 2) * 3 # [1] 36 #> ((10 + 2) * 3 - 6) / 3 # [1] 10
```

No R, assim como na matemática, os parênteses () servem para mudar a ordem natural das operações, indicando que se deve resolver primeiro o que está dentro deles, antes de seguir para o resto da expressão. Para um cientista social ou analista de políticas públicas, o R pode ser usado como uma super-calculadora que aplica uma conta a milhares de dados de uma só vez, de forma mais rápida e com menos chance de erros.

===== EXERCÍCIO 2 (10 pts)
=====

Objetos e nomeação (HOPR cap. 1)

ENTREGA: explique em 2–4 linhas a diferença entre ‘Name’ e ‘name’.

TODO: Execute e observe os resultados.

```
x <- 1:6 Name <- 1 name <- 0 Name + 1 name + 1
```

Para o R, N (maiúsculo) e n (minúsculo) são dois símbolos completamente diferentes e podem armazenar valores distintos. Isso significa que o R as trata como duas variáveis distintas.

===== EXERCÍCIO 3 (10 pts)
=====

Sorteio (sample) e reprodutibilidade (HOPR cap. 1–2)

ENTREGA: descreva o papel de set.seed() em 3–5 linhas.

TODO: Compare com e sem set.seed()

```
set.seed(123) dice <- 1:6 sample(dice, size = 2, replace = TRUE)
```

(Teste: reexecute as linhas acima e verifique se o resultado se repete)

(Agora remova ou mude o `set.seed` e compare)

A função `set.seed()` no R fixa o ponto de partida do gerador de números pseudoaleatórios. Ao fazer isso, ela garante que a mesma sequência de números seja produzida sempre que o código é executado, o que é fundamental para a reprodutibilidade. Esse controle torna as análises e simulações transparentes, verificáveis e colaborativas.

=====

===== EXERCÍCIO 4 (10 pts)
=====

Sua primeira função (HOPR cap. 1)

ENTREGA: explique o que faz cada linha da função em 4–6 linhas.

BÔNUS: implemente ‘`soma3()`’ (sorteia 3 números entre 1 e 6 e retorna a soma).

TODO: Defina a função e teste:

```
roll2 <- function(bones = 1:6) { dice <- sample(bones, size = 2, replace = TRUE) sum(dice) }  
#roll2 <- function(bones = 1:6): cria a função roll2 com um argumento bones, que tem como valor padrão  
um dado de seis faces (1:6); #dice <- sample(bones, size = 2, replace = TRUE): sorteia dois números do vetor  
bones (permitindo repetição para simular dois dados) e armazena o resultado na variável dice; #sum(dice):  
soma os dois números guardados em dice e retorna este total como o resultado final da função.
```

Testes sugeridos:

```
roll2() roll2(1:20)
```

(BÔNUS): implementar `soma3()`

```
soma3 <- function(bones = 1:6) dice <- sample(bones, size = 3, replace = TRUE) sum(dice)
```

=====

===== **EXERCÍCIO 5 (10 pts)** =====

Ajuda e exemplos (HOPR cap. 2)

ENTREGA: resuma argumentos de `sample()` e como consultar ajuda (4–6 linhas).

TODO: Consulte a ajuda e rode exemplos

`?sample`

`example(sample)`

`?sample example (sample)`

Dica: leia os argumentos ‘x’, ‘size’, ‘replace’, ‘prob’, etc.

#A função `sample()` do R sorteia amostras aleatórias de uma fonte de dados (x), que pode ser um vetor ou um número. Você define quantos itens quer (size), se permite repetições (`replace = TRUE`) e pode atribuir diferentes chances a cada item com um vetor de pesos (prob). #`example(“função”)`: para ver exemplos práticos executados diretamente no seu console.

=====

===== **EXERCÍCIO 6 (15 pts)** =====

Simulação e histograma (HOPR cap. 1–2)

ENTREGA: histograma, média, desvio-padrão; interpretação (4–6 linhas).

Dica: você pode salvar o gráfico com `png()... dev.off()`

```
set.seed(42) somas <- replicate(10000, roll2()) length(somas) hist(somas, main = “Soma de dois dados
(10.000 lançamentos)”, xlab = “Soma”) mean(somas) sd(somas)
```

#Inicialmente, para garantir resultados reprodutíveis, o código primeiro fixa a semente aleatória com `set.seed(42)`. Em seguida, ele executa a função `roll2()` (que soma o resultado de dois dados) 10.000 vezes, armazenando todas as somas em um vetor chamado `somas`. Por fim, `length(somas)` é usado como uma verificação simples para confirmar que o vetor contém os 10.000 resultados esperados da simulação. `#hist(somas)`: gera um gráfico que visualiza a distribuição das 10.000 somas. O gráfico confirma que 7 é o resultado mais frequente e que a distribuição geral se assemelha a uma “curva de sino”, como previsto pela teoria da probabilidade. `#mean(somas)`: calcula a média de todos os resultados, que se aproxima de 7.0 (6.9532). Isso demonstra a Lei dos Grandes Números, validando que a média teórica esperada para a soma de dois dados é 7. `#sd(somas)` mede a dispersão (o quão “espalhados” estão os resultados) em torno da média. O valor de 2.4329 confirma o desvio padrão teórico, quantificando a variabilidade típica dos lançamentos.

(Opcional) Salvar figura:

```
#png("ex6_hist_somas.png", width = 900, height = 600) # hist(somas, main = "Soma de dois dados
(10.000 lançamentos)", xlab = "Soma") # dev.off()
```

Extensão (opcional, sem pontos extras): dado viciado favorecendo o 6

```
prob_vies <- c(rep(1/8, 5), 3/8) somas_vies <- replicate(10000, sum(sample(1:6, size = 2, replace = TRUE,
prob = prob_vies))) hist(somas_vies, main = "Dado viciado (6 favorecido)", xlab = "Soma")
```

=====

===== EXERCÍCIO 7 (10 pts) =====

=====

Tipos básicos (HOPR cap. 3)

ENTREGA: explique o que ‘`str()`’ revela sobre cada tipo e cite um uso prático.

```
dbl <- c(1.5, 2.0) # numéricos (double) int <- c(1L, 2L) # inteiros chr <- c("saude","educacao") # texto
lgl <- c(TRUE, FALSE) # lógico
str(list(dbl = dbl, int = int, chr = chr, lgl = lgl))
```

#A função `str()` oferece uma verificação completa da estrutura e do tipo de cada variável. A saída revela, primeiramente, que a estrutura geral é uma lista, sendo certo que cada elemento tem uma natureza distinta. `#dbl`: identifica um vetor numérico com precisão decimal, apropriado para armazenar índices socioeconômicos (como IDH e Gini); `#int`: identifica um vetor de números inteiros, perfeito para dados de contagem, como populações e número de votos; `#chr`: identifica um vetor de texto, essencial para variáveis categóricas como siglas de partidos e níveis de escolaridade, por exemplo; `#lgl`: identifica um vetor lógico, cujos valores `TRUE` ou `FALSE` são cruciais para registrar, por exemplo, a elegibilidade em programas sociais, se um indivíduo votou ou não, e para criar filtros na base de dados.

=====

===== EXERCÍCIO 8 (10 pts) =====

Data.frame (baralho) e mini-base municipal (HOPR cap. 3)

ENTREGA: nº de linhas/colunas e breve interpretação do summary().

```
faces <- c("ace","two"," three "," four "," five ","six"," seven "," eight "," nine ","ten"," jack "," queen "," king ")
suits <- c(" spades "," hearts "," diamonds "," clubs ")
deck <- data.frame ( face = rep (faces , times = 4) , suit = rep (suits , each = 13) , value = rep (1:13 , times = 4) )
nrow ( deck ); ncol(deck); head (deck , 10) #serve para contar o núemro de linhas e colunas.
```

Mini - base municipal

```
set.seed (2025)
municipios <- paste0 ("Mun_", sprintf ("%02d", 1:10) )
dados_munic <- data.frame (
  municipio = municipios ,
  gasto_saude_pc = round ( runif (10 , 200 , 1200) , 2) ,
  taxa_evasao = round ( runif (10 , 0.00 , 0.20) , 3) ,
  taxa_desemprego = round ( rnorm (10 , 0.12 , 0.03) , 3) )
```

#runif - todos tem a mesma possibilidade de serem escolhidos.

```
head (dados_munic); #primeira fileira
summary (dados_munic)
```

#RESPOSTA: nrow(dados_munic) # = 10 ncol (dados_munic) # = 4 #summary - resumo dos dados estatísticos; mostra os indicadores mínimo, quartis, mediana, média e máximo;estes indicadores são úteis para identificar tendências centrais (média, mediana), dispersão (quartis, mínimo, máximo) e possíveis outliers. No caso do exercício, permite comparar rapidamente como os municípios variam em gasto de saúde, evasão escolar e desemprego.

=====

===== EXERCÍCIO 9 (10 pts) =====

Seleção e filtros (HOPR cap. 4)

ENTREGA: descreva os retornos e quantos municípios têm taxa_evasao > 0.10.

```
deck[1, ]
deck[c(1,3,5), c("face","suit")]
deck[-(1:48), ]
subset_hearts <- deck[ deck$suit == "hearts" ]
nrow(subset_hearts)
```



```

set.seed(2025) # definir a semente logo antes
municipios <- paste0("Mun_", sprintf("%02d", 1:10))
dados_munic <- data.frame( municipio = municipios, gasto_saude_pc = round(runif(10, 200, 1200), 2),
taxa_evasao = round(runif(10, 0.00, 0.20), 3), taxa_desemprego = round(rnorm(10, 0.12, 0.03), 3) )

evaz_alta <- dados_munic[dados_munic$taxa_evasao > 0.10, ]
nrow(evaz_alta)

#deck[1, ]: mostra a primeira linha do data frame (a primeira carta do baralho). #deck[c(1,3,5),
c("face","suit")]: apenas as colunas face e suit das linhas 1, 3 e 5 (três cartas específicas, mostrando apenas
o valor e o naipe). #deck[-(1:48), ]: retorna todas as linhas exceto da 1 até a 48. Se o baralho tiver 52
cartas, retorna apenas as últimas 4 cartas. #subset_hearts <- deck[ deck$suit == "hearts"]: cria um
subconjunto contendo apenas as cartas de naipe "hearts". # nrow(subset_hearts): retorna o número de
linhas desse subconjunto, ou seja, quantas cartas de copas existem no baralho.

#set.seed(2025): semente fixa #evaz_alta <- dados_munic[ dados_munic$taxa_evasao > 0.10, ]: cria um
subconjunto com os municípios cuja taxa de evasão escolar é maior que 0.10 (10%). #evaz_alta: mostra os
dados desses municípios (nome, gasto em saúde, taxa de evasão e taxa de desemprego).

#SÃO 4 MUNICÍPIOS com taxa_evasao > 0.10: (MUN_02, 03, 06, 09).

```

=====

===== EXERCÍCIO 10 (10 pts) =====

=====

Modificando valores e NA (HOPR cap. 5)

ENTREGA: explique o efeito de `na.rm = TRUE` e quando usá-lo.

Modificando valores (ases = 14)

```
deck2 <- deck #fazendo uma cópia
deck2$value[c(13, 26, 39, 52)] <- 14
head(deck2, 13)
```

Valores ausentes

```

vals <- c(NA, 1:5)
mean(vals)
mean(vals, na.rm = TRUE)

dados_m2 <- dados_munic
dados_m2$taxa_evasao[3] <- NA
dados_m2$gasto_saude_pc[7] <- NA

mean(dados_m2$taxa_evasao)
mean(dados_m2$taxa_evasao, na.rm=TRUE)

#como o R não sabe como lidar com a ausência de dados, para calcular uma média mean() quando houver
valores ausentes, é preciso usar o na.rm = TRUE; assim, o R ignora os valores ausentes e calcula a média
apenas com os valores disponíveis; isso é útil quando se quer obter uma estatística descritiva mesmo que
haja dados faltando.

```

=====

===== EXERCÍCIO 11 (OPCIONAL, até 10 pts) =====

Funções que "guardam estado" (HOPR cap. 6)

ENTREGA: explique o conceito e dê exemplo análogo em PP/CS.

```
setup <- function(deck_init) { DECK <- deck_init # cópia interna (estado)

DEAL <- function() { # Devolve a primeira carta e atualiza o baralho interno removendo-a. card <-
DECK[1, , drop = FALSE] DECK «- DECK[-1, , drop = FALSE] return(card) }

SHUFFLE <- function() { # Reembaralha o baralho interno idx <- sample(seq_len(nrow(deck_init)), size
= nrow(deck_init)) DECK «- deck_init[idx, , drop = FALSE] invisible(NULL) }

list(deal = DEAL, shuffle = SHUFFLE) }

cards <- setup(deck) cardsdeal(); cardsdeal(); cardsshuffle(); cardsdeal()
```

A função setup() cria uma cópia interna do baralho que só pode ser manipulada pelas funções definidas dentro dela.

#DEAL() tira a primeira carta e SHUFFLE() embaralha o baralho. #O objeto cards expõe apenas essas operações, sem permitir acesso direto ao baralho (abstração e controle de acesso). #Em políticas públicas, isso pode ser comparado a um programa de vacinação: o estoque de vacinas existe internamente, mas não é acessado pela população; as regras do programa funcionam como as funções, permitindo apenas aplicar doses ou repor o estoque. Dessa forma, o sistema garante organização e evita que o recurso seja manipulado diretamente.

=====

===== EXERCÍCIO 12 (15 pts) =====

Mini-projeto integrador: Saúde

ENTREGA: 2 histogramas, diferença de médias e interpretação (6–8 linhas).

```
set.seed(123) pressao_saude <- function() { demanda <- sample(1:6, 1, TRUE) equipe <- sample(1:6, 1,
TRUE) insumos <- sample(1:6, 1, TRUE) demanda + equipe + insumos } prs <- replicate(10000, pres-
sao_saude()) hist(prs, main = "Pressão no sistema de saúde (simulada)", xlab = "Índice") mean(prs);
sd(prs)
```

Viés em demanda (favorece 6)

```
prob_demanda <- c(rep(1/8, 5), 3/8) pressao_vies <- function() { demanda <- sample(1:6, 1, TRUE, prob
= prob_demanda) equipe <- sample(1:6, 1, TRUE) insumos <- sample(1:6, 1, TRUE) demanda + equipe +
insumos } prs_bias <- replicate(10000, pressao_vies()) hist(prs_bias, main = "Pressão no sistema de saúde
(com viés)", xlab = "Índice") mean(prs_bias) - mean(prs)
```

(Opcional) Salvar figuras:

```
png("ex12_hist_semvies.png", width = 900, height = 600);
hist(prs); dev.off()
```

```
png("ex12_hist_comvies.png", width = 900, height = 600);
hist(prs_bias); dev.off()
```

#Temos 2 distribuições simuladas da “pressão no sistema de saúde”: #um histograma sem viés que mostra a soma das variáveis demanda, equipe e insumos sorteadas de forma uniforme entre 1 e 6. Como resultado, a distribuição é centrada em torno de 10–11. #um histograma com viés que favorece valores altos de demanda (probabilidade maior para 6). Como resultado, a distribuição é deslocada para a direita, aumentando a frequência de índices mais elevados. # a diferença de médias é de 0,6377, indicando que o viés em demanda eleva o índice médio de pressão.

=====

===== ENCERRAMENTO / CHECK-
LIST =====

- O script roda do início ao fim sem erros?
- Há comentários explicando seus passos e interpretações no PDF?
- Você usou `set.seed()` nas simulações?
- Nomeou os arquivos conforme instruções do professor?

Informações da sessão (útil para reprodutibilidade)

```
sessionInfo() # ===== FIM DO SCRIPT =====
```